

INSAT

Architectures Réactives et Microservices

TP1: Design Patterns — Refactoring d'une application Java

Durée	1h30	Langage	Java
Thème	Design Patterns	Travail	Individuel / binôme

1. Objectifs du TP

- Identifier des problèmes de conception dans un code existant.
- Choisir un Design Pattern en fonction du problème rencontré.
- Refactoriser progressivement une application Java.
- Comprendre les rôles des classes et leurs interactions dans un pattern.
- Justifier les choix de conception et leurs conséquences.

Important : le nom du pattern n'est pas toujours donné. Vous devez d'abord analyser le problème, puis proposer une solution.

2. Contexte

On considère une application Java simplifiée permettant de gérer les produits et les commandes d'une boutique en ligne. L'application fonctionne, mais sa conception présente plusieurs problèmes : création directe des objets, dépendances fortes entre classes, multiplication des conditions if/else, difficulté d'ajouter de nouveaux comportements et absence de mécanisme de notification.

3. Partie 1 — Analyse du code existant

Étudier le code suivant avant toute modification.

```
public class Product {

    private String type;
    private String name;
    private double price;

    public Product(String type, String name, double price) {
        this.type = type;
        this.name = name;
        this.price = price;
    }

    // getters...
}

public class OrderService {

    public void createOrder(String type, String name, double price) {

        Product product;
```

```

        if (type.equals("BOOK")) {
            product = new Product("BOOK", name, price);
        }
        else if (type.equals("ELECTRONIC")) {
            product = new Product("ELECTRONIC", name, price);
        }
        else if (type.equals("CLOTHING")) {
            product = new Product("CLOTHING", name, price);
        }
        else {
            throw new IllegalArgumentException("Unknown product");
        }

        System.out.println("Order created for " + product.getName());
    }
}

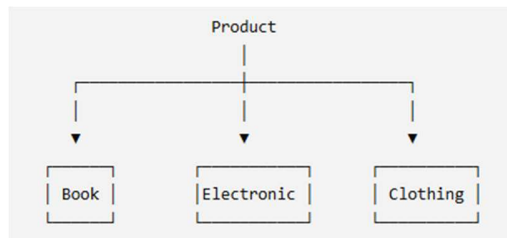
```

Travail demandé :

1. Identifier au moins trois problèmes de conception.
2. Expliquer pourquoi l'ajout d'un nouveau type de produit rend le code plus difficile à maintenir.
3. Identifier le principe de conception qui est mis à mal.
4. Proposer un Design Pattern permettant d'améliorer la création des produits.
5. Justifier votre choix.

4. Partie 2 — Refactoring de la création des produits

On souhaite obtenir une architecture permettant de créer plusieurs types de produits :



Travail demandé :

1. Créer l'interface Product contenant la méthode display().
2. Créer les classes Book, Electronic et Clothing.
3. Créer une classe responsable de la création des produits.
4. Modifier OrderService afin qu'il ne crée plus directement les objets avec new.
5. Ajouter un nouveau produit Food sans modifier la logique de OrderService.
6. Tester plusieurs créations de produits.

Exemple d'utilisation attendu :

```

Product p = ProductFactory.createProduct("BOOK", "Design Patterns", 45);
p.display();

```

7. pourquoi cette solution est-elle plus flexible que la création directe des objets ?

5. Partie 3 — Instance unique pour la configuration

```
public class ApplicationConfig {  
  
    private String databaseUrl;  
    private String applicationName;  
  
    public ApplicationConfig() {  
    }  
  
    // getters / setters  
}
```

Plusieurs composants peuvent actuellement créer leur propre configuration. Modifier cette classe afin de garantir une instance unique.

```
ApplicationConfig c1 = ApplicationConfig.getInstance();  
ApplicationConfig c2 = ApplicationConfig.getInstance();  
  
System.out.println(c1 == c2);
```

Travail demandé :

1. Modifier le constructeur et l'accès à l'instance.
2. Implémenter getInstance().
3. Vérifier expérimentalement que c1 et c2 désignent le même objet.
4. Expliquer dans quel contexte ce pattern est pertinent.

6. Partie 4 — Intégration d'un ancien système de paiement

```
public interface PaymentService {  
    void pay(double amount);  
}  
  
public class OldPaymentSystem {  
  
    public void makePayment(double value) {  
        System.out.println("Payment : " + value);  
    }  
}
```

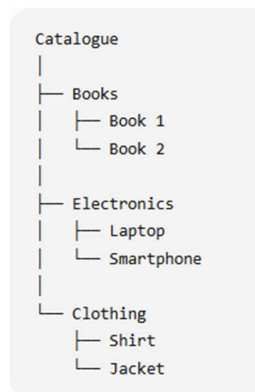
Le client attend PaymentService, mais l'ancien système fournit makePayment(). Les deux interfaces sont incompatibles.

Travail demandé :

1. Créer une classe intermédiaire permettant au client d'utiliser OldPaymentSystem via PaymentService.
2. Le code client doit pouvoir rester indépendant de l'interface de l'ancien système.
3. Tester un paiement de 250.
4. Expliquer pourquoi il est préférable de ne pas modifier OldPaymentSystem.

```
PaymentService payment =  
    new PaymentAdapter(new OldPaymentSystem());  
  
payment.pay(250);
```

7. Partie 5 — Construire le catalogue sous forme d'arbre



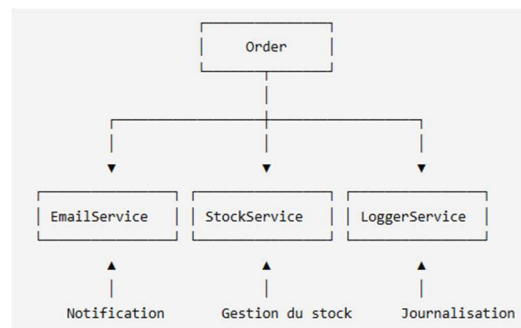
L'objectif est de pouvoir traiter un produit individuel et une catégorie de produits de manière uniforme.

Travail demandé

1. Créer une abstraction commune aux éléments du catalogue.
2. Créer Product pour représenter un élément individuel.
3. Créer Category pour représenter une composition de plusieurs éléments.
4. Permettre à une catégorie de contenir des produits et/ou d'autres catégories.
5. Implémenter display() afin d'afficher toute l'arborescence.
6. Tester l'affichage à partir de l'objet Catalogue.

8. Partie 6 — Notification des changements d'état

Lorsqu'une commande change d'état, plusieurs composants doivent être informés automatiquement.



Travail demandé :

6. Créer une abstraction Observer avec update(String status).
7. Créer une abstraction Subject permettant d'ajouter, supprimer et notifier les observateurs.
8. Faire de Order le sujet observé.
9. Créer EmailService, StockService et LoggerService comme observateurs.
10. Lors d'un changement d'état, notifier automatiquement tous les observateurs.
11. Tester le passage de l'état CREATED à SHIPPED.

```
Order order = new Order();

order.attach(new EmailService());
order.attach(new StockService());
order.attach(new LoggerService());

order.setStatus("SHIPPED");
```

Question : quels problèmes apparaîtraient si Order appelait directement emailService.update(), stockService.update(), etc. ?

9. Partie 7 — Challenge : choisir le pattern adapté

Cette partie évalue votre capacité à identifier un problème de conception sans que le pattern soit explicitement indiqué.

```
public class NotificationService {  
  
    public void send(String type, String message) {  
  
        if (type.equals("EMAIL")) {  
            System.out.println("Sending Email : " + message);  
        }  
        else if (type.equals("SMS")) {  
            System.out.println("Sending SMS : " + message);  
        }  
        else if (type.equals("PUSH")) {  
            System.out.println("Sending Push Notification : " + message);  
        }  
    }  
}
```

Le responsable souhaite pouvoir ajouter facilement de nouveaux mécanismes de notification sans modifier le code principal.

Travail demandé :

1. Identifier le problème de conception.
2. Choisir le Design Pattern le plus approprié et justifier votre choix.
3. Refactoriser le code.
4. Ajouter WhatsAppNotification sans modifier la classe principale.
5. Comparer l'ancienne et la nouvelle architecture tout en justifiant vos choix et en précisant comment cette architecture améliore la modularité, le couplage et la maintenabilité.
6. Réaliser un diagramme d'architecture montrant les principales couches, les composants et le sens des dépendances.
7. Indiquer comment les Design Patterns étudiés (Factory, Singleton, Adapter, Composite et Observer) peuvent s'intégrer dans cette architecture.

Questions:

Pourquoi est-il préférable de ne pas considérer un Design Pattern comme une architecture complète, mais comme une solution à un problème de conception particulier ?

10. Partie 8 — Question d'architecture

L'architecture actuelle contient les classes métier, les services et les mécanismes de création/notification dans un même niveau. Le responsable souhaite obtenir une architecture plus claire, où les responsabilités sont mieux séparées et où les Design Patterns étudiés peuvent être intégrés sans créer de dépendances inutiles.

On souhaite faire évoluer l'application afin qu'elle puisse être utilisée par plusieurs types de clients : une application Web, une application mobile et éventuellement d'autres applications clientes.

De façon Générale, Indiquer dans quelle couche placer les éléments suivants : Product, ProductFactory, Order, OrderService, PaymentAdapter, EmailService et ApplicationConfig. Proposer une architecture en couches pour organiser l'application.

11. Travail à rendre

- Code source Java correctement organisé.
- Diagramme UML de la solution finale.
- Une courte justification de chaque Design Pattern utilisé.
- Tests permettant de démontrer le fonctionnement des solutions.
- Une comparaison entre la conception initiale et la conception refactorisée.

Bon travail !